FR

Bienvenue sur Sakata

Ce portail est un sanctuaire numérique dédié à la préservation et à la transmission de l'identité culturelle du peuple **Sakata**.

Notre objectif est d'unir nos racines ancestrales avec la splendeur numérique moderne, en offrant à la fois des archives vivantes, des sagesses intemporelles et un forum d'échange intergénérationnel.

Suivant →

> *« Mboka » signifie « le village » en kisakata. Le forum est conçu comme la place publique du village numérique : ouvert mais ordonné, vivant mais respectueux. Sa complexité technique vient de la coexistence de trois modes — fil long terme, chat temps réel, réactions instantanées — sur la même infrastructure.*

DÉFINITION

Qu'est-ce que c'est ?

Accessible via `/forum`, Mboka organise les discussions en trois niveaux :

- **Catégories** (Histoire, Langue, Musique, Cuisine, Diaspora, Actualité) — fixes, non créables par les users.
- **Fils (threads)** — discussions thématiques créables par tout user authentifié.
- **Messages** — au sein d'un fil, échanges en temps réel avec réactions et mentions.

Chaque fil a un éditeur Markdown personnalisé pour le post initial (riche), puis dégénère en chat texte simple pour les réponses (rapide).

TABLES

Schéma de données

TABLE	RÔLE	PARTICULARITÉ
`forum_categories`	Catégories fixes	Slug, icon, couleur, ordre — seed initial
`forum_threads`	Fils de discussion	Slug auto-généré + verrouillage par auteur ou modo
`forum_messages`	Messages dans un fil	Markdown nettoyé côté client + serveur
`forum_reactions`	Réactions emoji	M:N user × message × emoji (unique constraint)
`forum_typing`	Présence de frappe	Broadcast-only (pas persisté)

DÉCISIONS

Pourquoi cette structure

Catégories fixes (et non user-generated)

CHOIX

Les catégories sont définies par seed et non créables dynamiquement.

Les forums où n'importe qui crée une catégorie deviennent des marécages : doublons, catégories vides, pollution. Six catégories fixes, claires, suffisent à couvrir tous les sujets attendus. Si le besoin émerge plus tard, il sera plus simple d'ajouter une 7e que de purger 50 doublons mal triés.

Markdown personnalisé pour les posts initiaux

On utilise un sous-ensemble strict de Markdown : **gras**, *italique*, citations, listes, liens, images. Pas de HTML brut, pas d'iframe, pas de scripts. Sanitisation via `DOMPurify` côté client + validation longueur/contenu côté serveur.

ANTI-XSS

Tout texte contenant `<script>`, `<iframe>`, ou des handlers `onerror=` est rejeté à l'insertion.

RLS par catégorie

CODE SQL

```sql
-- Lecture : tous les users authentifiés peuvent lire
CREATE POLICY "forum_messages_read"
  ON forum_messages FOR SELECT
  TO authenticated USING (true);

-- Écriture : seul l'auteur de son propre message
CREATE POLICY "forum_messages_insert"
  ON forum_messages FOR INSERT
  TO authenticated
  WITH CHECK (auth.uid() = author_id);

-- Modification : auteur (5 min) OU manager+ (toujours)
CREATE POLICY "forum_messages_update"
  ON forum_messages FOR UPDATE
  TO authenticated
  USING (
    (auth.uid() = author_id AND created_at > NOW() - INTERVAL '5 minutes')
    OR EXISTS (
      SELECT 1 FROM profiles
      WHERE profiles.id = auth.uid()
        AND profiles.role IN ('admin', 'manager')
    )
  );
```

PATTERN

Realtime — broadcast vs postgres_changes

Pour les messages : postgres_changes

Les messages sont persistés. Subscription Supabase Realtime sur la table `forum_messages` filtrée par `thread_id`. Garantit que tout client connecté voit les nouveaux messages en < 200ms.

Pour la frappe : broadcast

La frappe est éphémère par nature. Pas besoin de persister. Le canal `thread:typing:{thread_id}` diffuse les events sans écrire en DB → 10× moins de pression sur la base.

CODE TYPESCRIPT

```typescript
channel.send({
  type: "broadcast",
  event: "typing",
  payload: { user_id: userId, name: profile.username }
});
```

Pour les réactions : optimistic UI

Quand un user clique 👍 sur un message, on met à jour l'UI immédiatement avant la confirmation serveur. Si l'INSERT échoue (rare, RLS satisfaite), on rollback avec un toast d'erreur.

OUTILS

Modération

- **Verrouillage de fil** — bouton « Verrouiller » dans `/admin/forum`, empêche nouveaux messages.
- **Soft-delete de message** — flag `deleted_at`, affichage « [Message supprimé] » mais audit conservé.
- **Bannissement** — flag `profiles.is_banned`, RLS interdit insert sur tables forum.
- **Signalement user-driven** — bouton « Signaler » crée un ticket dans `moderation_reports`.

RETOUR D'EXPÉRIENCE

Difficultés rencontrées

Sérialisation Next.js des icônes lucide

Les composants serveur ne peuvent pas sérialiser les icônes Lucide passées comme prop. Solution : passer le **nom** de l'icône en string, puis mapper vers le composant côté client.

Subscription qui ne reçoit jamais d'event

Symptôme : nouveau message en DB visible dans Studio Supabase, mais le client ne le voit pas en temps réel. Cause : la table n'avait pas la **publication realtime activée**.

CODE SQL

```sql
ALTER PUBLICATION supabase_realtime
  ADD TABLE forum_messages, forum_reactions;
```

À SAVOIR

Limitations connues

- Pas de threading dans un message (pas de réponse imbriquée).
- Pas de notifications push pour les mentions (prévu en roadmap PWA).
- Recherche dans les messages limitée à ilike — pas de full-text search Postgres pour l'instant.

Sakata.com — Documentation interne · feature-forum-mboka · v2026.4

« Moto oyo ayebi te kotombola miso na likolo, akoki te koyeba esika oyo mbula eutaka. » — Proverbe Sakata

SAKATA

© 2026 — Brume de la Rivière. Tous droits réservés.

CULTURE ECOLE LANGUE HISTOIRE FORUM